Tutorial for GIS analysis

Nourat Noemi Alazza, Marie Horn

2026-02-14

How to analyse spatial data in public health context

Using the example of tobacco outlets near schools in Bremen, Germany

This document shows an easy-to-follow, reproducible GIS workflow in R to study tobacco outlets near schools in the German city Bremen. We use spatial data (geolocation coordinates) of outlets that sell cigarettes, including vending machines and shops such as supermarkets and kiosks. In addition, we use the locations of schools in the city of Bremen to analyze how many tobacco outlets are located within a 500-meter radius of each school. The main research question is *whether schools in socially disadvantaged areas are surrounded by more tobacco outlets than schools in more privileged areas.* We also account for population density of the surrounding area as a potential confounding factor.

Part 1: Project paths

First of all, we have to create the project structure. This script assumes the following structure:

1. `gis_tobacco`
    a. `project_data_champion_award`
        - `outputs`
        - `scripts`
        - `sources`
        - `Tutorial_GIS.pdf`
        - `Tutorial_GIS.Rmd`
    b. `Tobacco_schools.Rproj`

If you change the structure of the folders or the location of the files, you also have to adapt the paths in this script!

To set up the project in R, we create a new R project called "Tobacco_schools.Rproj" in the "gis_tobacco" folder within the integrated development environment (RStudio or VS Code). Next, we copy the "project_data_champion_award" folder and all relevant files into the "gis_tobacco" folder.

Then, we open the R script "data_analysis" to view and practice with the code. When the script is open, we always have to first load the R-packages needed.

```r
library(here)     # creates relative paths for the R project to access the data
library(sf)       # encodes and analyzes spatial vector data
library(readr)    # reads delimited and CSV files
library(dplyr)    # edits data
library(tidyr)    # edits data
library(tmap)     # creates maps
library(osmdata)  # gets data from OpenStreetMap
library(ggplot2)  # creates graphics and illustrations
library(stringr)  # converts strings
library(scales)   # defines and formats numeric/categorial scales for graphics
library(glmmTMB)  # runs generalized linear mixed models
```

After that, we define the paths where the source and target files are stored. With relative project paths the code remains portable across devices and team members. To make sure the paths work and we have access to the files, a "stop" function is added. This script uses the "here" package to build file paths. It automatically sets the project root to the folder where the R-project (.Rproj) file is located.

```r
file_school_data  <- here("project_data_champion_award",
                          "sources",
                          "school_data",
                          "schools.shp")
file_machine_data <- here("project_data_champion_award",
                          "sources",
                          "cig_vending_machines.csv")
file_shop_data    <- here("project_data_champion_award",
                          "sources",
                          "cig_shop_data.csv")

stopifnot(file.exists(file_school_data), file.exists(file_shop_data))
```

Part 2: How do we get the data with locations?

There are different ways to get geographic data. 1. The data can be collected by oneself, e.g. with a smartphone-app. 2. Open sources like OpenStreetMap or sites by the government offer spatial data for free. 3. You can buy chunks of data about the aspect of interest by companies like Google Maps.

The data come from different sources. We use publicly available data from the portal metaver.de to obtain the locations of schools (as a shapefile). For the locations of cigarette-selling retail outlets, we purchased data from Google Maps via Apify (as a CSV file). We chose to rely on Google Maps data because we have not found any government agency or other institution that was willing or able to provide us with data on tobacco outlets and OpenStreetMap may contain more missing entries and often provides less information about the objects, which makes data cleaning more time-consuming. Although purchasing Google Maps data is more expensive, it reduces the amount of missing information and provides more information about the objects, which makes cleaning and processing the data easier. If you need help collecting the cigarette-selling retail outlets via Apify, please contact us. We've only uploaded a sample of the data to this repository.

To get the locations of the cigarette vending machines we use a short R code to get the data from OpenStreetMap. To do that, first we have to define the bounding box using geographic coordinates. *The bounding box specifies the area we are interested in*, in this case the city of Bremen. If you do not know the bounding box for your area of interest, you can also ask OpenStreetMap and Nominatim for it. Then you should always check whether the result actually covers the full city. Here, "x" stands for the longitude and "y" for latitude. Here is an example:

```r
city_bbox <- getbb("London, England", format_out = "matrix")
print(city_bbox)
```

```
##          min        max
## x -0.5103751  0.3340155
## y 51.2867601 51.6918741
```

Next, we build an Overpass API query using the R-package "osmdata" (Padgham et al., 2017). You have to enter the coordinates of the bounding box and the key words under which the item of interest is stored in OSM. To find out which key words are relevant for your research question, you can look up the item you want to map on the OSM Wiki "https://wiki.openstreetmap.org/". Once you run your query you can extract the point data and put them into a data frame.

```r
# Bremen city bbox (lon_min, lat_min, lon_max, lat_max)
bremen_bbox <- c(8.48, 53.01, 8.98, 53.23)

# Build and run Overpass query
query <- opq(bbox = bremen_bbox) %>%
  add_osm_feature(key = "amenity", value = "vending_machine") %>%
  add_osm_feature(key = "vending",  value = "cigarettes", value_exact = FALSE)

machine_query  <- osmdata_sf(query)
machine_points <- machine_query$osm_points

coords <- st_coordinates(machine_points)
machine <- data.frame(
  id  = machine_points$osm_id,
  lon = round(coords[, 1], 5),
  lat = round(coords[, 2], 5)
)

# Save as CSV
write.table(
  machine,
  file = file_machine_data,
  sep = ";",
  dec = ".",
  row.names = FALSE
)
```

Now we bring all the data together: we load the data of the schools, machines and shops, and we put each object into the coordinate system that fits its use case. In order to use the data in maps we transform it into the coordinate system *WGS84* with the code *EPSG 4326*. As soon as we want to compute distances (for example a 500 meter-buffer around objects) we work in *ETRS89* with the code *EPSG 25832* so that the coordinates are in meters. As a small memory aid we use the endings *"_map"* for mapping objects and *"_met"* for objects that are used for metric analysis.

```r
# Transform school data (from shapefile)
school_raw <- st_read(file_school_data, quiet = TRUE) %>%
  rename(school_type = schulart_2) # change variable name to English
school_met <- school_raw %>%
  mutate(school_id = row_number()) %>%
```

```
  rename(school_name = nam,
         school_neighborhood = ortsteilna)

# Quick check: is school_met in the correct crs?
if (is.na(st_crs(school_met)$epsg) || st_crs(school_met)$epsg != 25832) {
  school_met <- st_transform(school_met, 25832)}

school_map <- st_transform(school_met, 4326)

# Transform machine data (from CSV file)
machine_raw <- read_delim(file_machine_data, delim = ";",
                          locale = locale(decimal_mark = "."),
                          show_col_types = FALSE)
machine_map <- st_as_sf(machine_raw, coords = c("lon","lat"), crs = 4326)
machine_met <- st_transform(machine_map, 25832)

# Transform shop data (from CSV file)
shop_raw <- read_csv(file_shop_data, show_col_types = FALSE) %>%
  mutate(lon = as.numeric(lon), lat = as.numeric(lat))
shop_map <- st_as_sf(shop_raw, coords = c("lon","lat"), crs = 4326)
shop_met <- st_transform(shop_map, 25832)
```

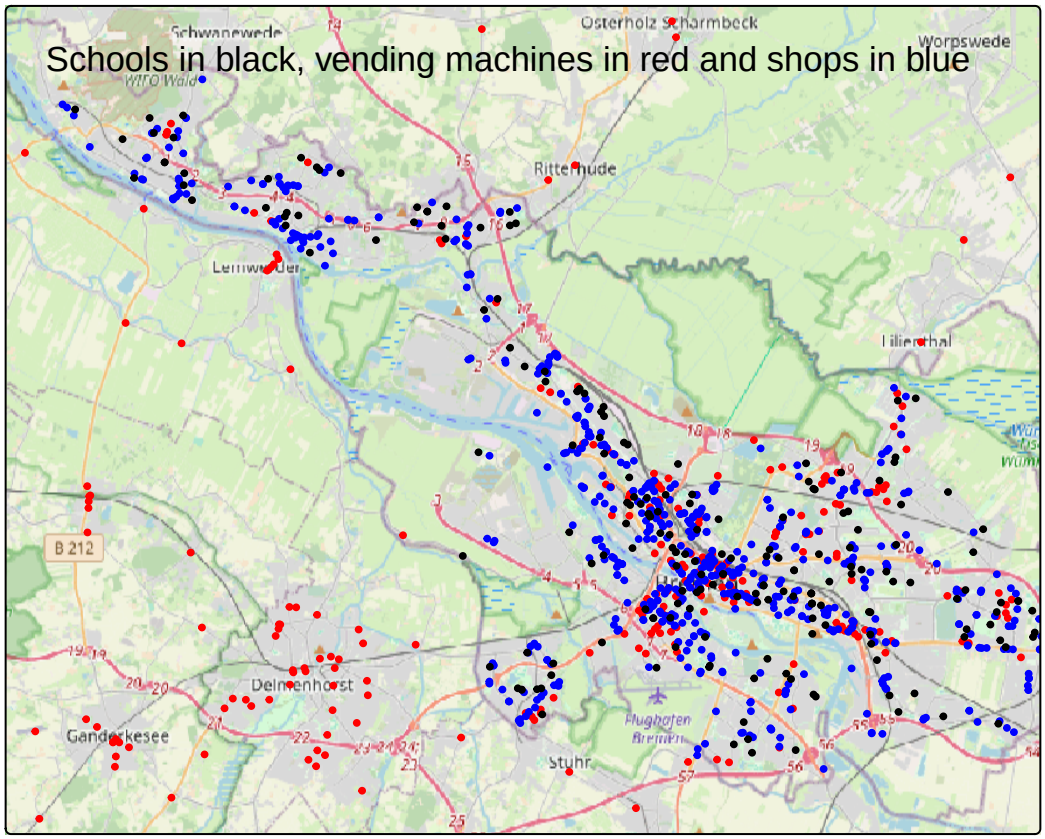
Part 3: Making the maps

In this part we draw the data that we prepared earlier. Using the R-package "tmap" (Tennekes, 2018), we begin with a simple map that shows the locations of the vending machines, the retail shops, and the schools. After that we create buffers with a radius of 500 meters around each school, filter the outlets that fall inside these buffers, and we draw a focused map that shows only what is relevant for the research question. If you are following the script, the map output is interactive because the setup chunk set the tmap mode to view, therefore you can pan and zoom like in a web map.

3.1 Location of tobacco outlets and schools in Bremen

This view helps you to build an intuition for the spatial pattern. You can zoom into a neighborhood and see how schools and outlets are arranged.

```
tm_basemap("OpenStreetMap") +
  tm_shape(machine_map) + tm_dots(fill = "red",   size = 0.2) +
  tm_shape(shop_map)    + tm_dots(fill = "blue",  size = 0.2) +
  tm_shape(school_map)  + tm_dots(fill = "black", size = 0.2) +
  tm_layout(title = "Schools in black, vending machines in red and shops in blue")
```



3.2 500-meter buffers around schools

For distance based questions we need metric coordinates. We therefore create the buffers with the objects that live in EPSG25832 (using the variables with the ending "_met"). We then transform the results back to WGS84 ("_map") for drawing on a web map. The filter keeps only the outlets that are within the buffers, and the final map shows exactly that.

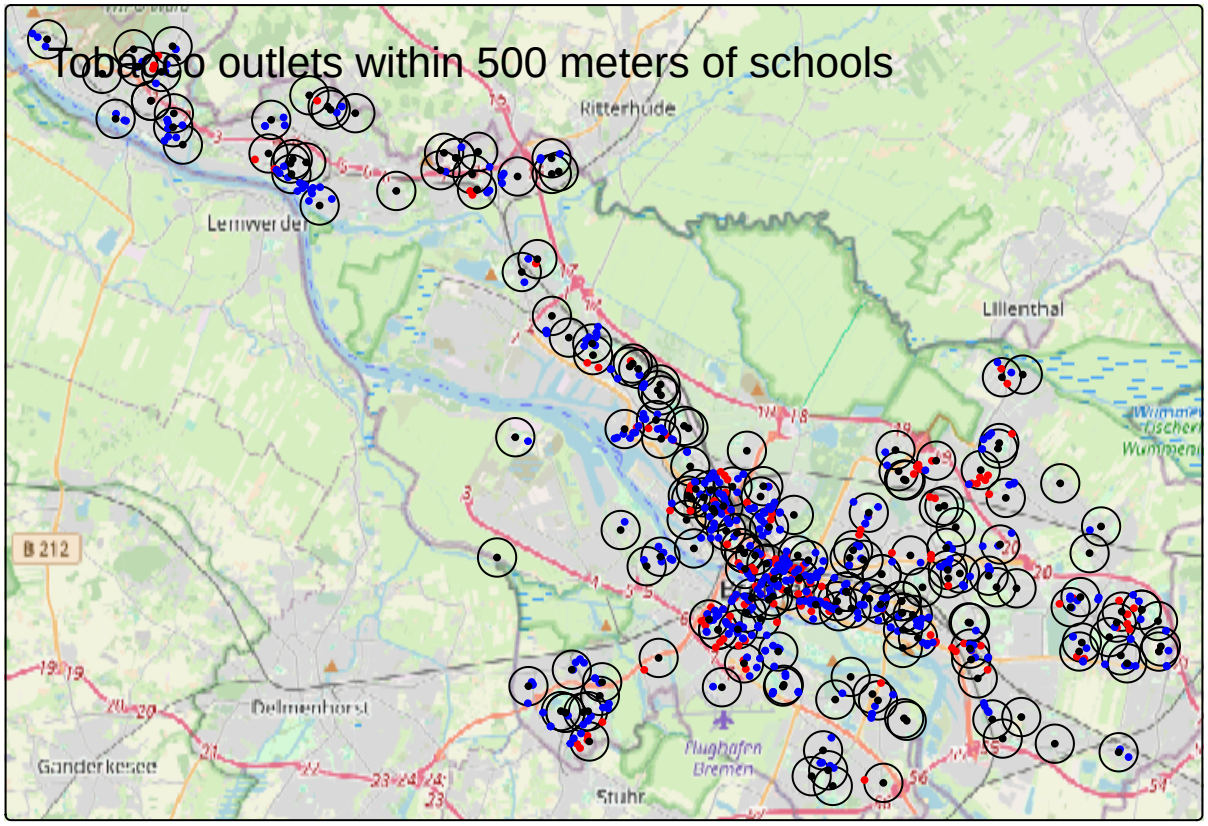
```
# Buffers in meters
school_buffer_500 <- st_buffer(school_met, 500)

# Intersect outlets with the buffers, still in meters
machine_500_met <- st_filter(machine_met, school_buffer_500)
shop_500_met    <- st_filter(shop_met,    school_buffer_500)

# Transform everything to WGS84 for web mapping
school_buffer_500_map <- st_transform(school_buffer_500, 4326)
machine_500_map       <- st_transform(machine_500_met,   4326)
shop_500_map          <- st_transform(shop_500_met,      4326)

# Create the map with the buffers
tm_basemap("OpenStreetMap") +
  tm_shape(school_buffer_500_map) + tm_borders(col = "black") +
  tm_shape(machine_500_map) + tm_dots(fill = "red",  size = 0.2) +
  tm_shape(shop_500_map) + tm_dots(fill = "blue", size = 0.2) +
  tm_shape(school_map) + tm_dots(fill = "black", size = 0.2) +
```

```
  tm_layout(title = "Tobacco outlets within 500 meters of schools")
```



Part 4: Descriptive summaries and one simple plot

In this part we count how many outlets are near each school. We then summarise these counts by school type, so that we can see typical values and the spread in a way that is easy to read. We then build one compact bar chart that shows the average number of vending machines and retail shops per school type.

4.1 Combine machines and shops into one outlet layer

We first create a single object that contains both vending machines and retail shops. This makes the next steps shorter and easier to read. Machines do not have a shop category, therefore we label their main category as “machine”, while shops keep their existing main category from the original data (like “supermarkets”, “kiosks”, “gas station”).

```
machines_met2 <- machine_met %>%
  mutate(
    outlet_id     = as.character(id),
    type          = "machine",
    main_category = "machine"
  )

shops_met2 <- shop_met %>%
```

```
  mutate(outlet_id = as.character(placeId),
         type      = "shop")

outlets_met <- bind_rows(machines_met2, shops_met2) %>%
  select(outlet_id, type, main_category, geometry)
```

If you are curious about the overall composition in Bremen, you can count how many records there are by outlet type and by main category and then print the result.

```
bremen_total <- outlets_met %>%
  st_drop_geometry() %>%
  count(type, name = "n_total_type") %>%
  arrange(desc(n_total_type))
print(bremen_total)
```

```
## # A tibble: 2 x 2
##   type    n_total_type
##   <chr>          <int>
## 1 shop             639
## 2 machine          262
```

4.2 Outlets within 500 meters of schools

We now keep only those outlets that fall inside the 500-meter buffers around schools. This step uses the objects in meters that we created earlier, because distance based operations require a metric coordinate system.

```
buffer_outlets <- st_join(
  school_buffer_500 %>% select(school_id, school_type, school_name,
                               school_neighborhood),
  outlets_met,
  join = st_intersects,
  left = TRUE
)
```

4.3 Counts per school

For each school, we count how many vending machines and how many shops are located within 500 meters. We also compute the total and attach a few school attributes so the table is self-explanatory. Because school systems differ across countries, we keep the original German school types used in Bremen. To help with the interpretation, here are English translations for the terms: - Berufsbildende Schule → Vocational school (or vocational training school) - Erwachsenenschule → Adult education center - Förderzentrum → Special needs/special education school - Grundschule → Primary school - Gymnasium → Academic secondary school - Oberschule → Comprehensive secondary school

```
counts_per_school <- buffer_outlets %>%
  st_drop_geometry() %>%
  group_by(school_id, school_type, school_name, school_neighborhood) %>%
  summarise(
    n_machines = sum(type == "machine", na.rm = TRUE),
    n_shops    = sum(type == "shop",    na.rm = TRUE),
```

```
    n_total    = n_machines + n_shops,
    .groups    = "drop"
  ) %>%
  arrange(school_id)

head(counts_per_school)
```

```
## # A tibble: 6 x 7
##   school_id school_type school_name       school_neighborhood n_machines n_shops
##       <int> <chr>       <chr>             <chr>                     <int>   <int>
## 1         1 Grundschule Schule an der Ad~ Findorff-Bürgerwei~           4      14
## 2         2 Grundschule Schule am Alten ~ Hastedt                       0       1
## 3         3 Grundschule Schule Am Mönchs~ Lesum                         2       2
## 4         4 Grundschule Schule An der Ge~ Gete                          0       2
## 5         5 Grundschule Schule an der Al~ Kattenesch                    0       2
## 6         6 Grundschule Schule Arbergen   Arbergen                      0       0
## # i 1 more variable: n_total <int>
```

If you want to save this table for use in other tools, you can export it as a CSV file.

```
write_csv(counts_per_school, here("project_data_champion_award",
                                  "outputs",
                                  "counts_per_school.csv"))
```

4.4 Summary by school type

To get more descriptive statistics on our data, we compute the mean, standard deviation (rounded to two decimals) and a few percentiles of the number of outlets (in total) near the schools in Bremen.

```
stats_by_schooltype <- counts_per_school %>%
  group_by(school_type) %>%
  summarise(
    mean_total   = round(mean(n_total,  na.rm = TRUE),2),
    sd_total     = round(sd(n_total,    na.rm = TRUE),2),
    min_total    = min(n_total,   na.rm = TRUE),
    p25_total    = stats::quantile(n_total, 0.25, na.rm = TRUE),
    median_total = stats::quantile(n_total, 0.50, na.rm = TRUE),
    p75_total    = stats::quantile(n_total, 0.75, na.rm = TRUE),
    max_total    = max(n_total,   na.rm = TRUE),
    n_schools    = n(),
    .groups      = "drop"
  ) %>%
  arrange(school_type)

head(stats_by_schooltype)
```

```
## # A tibble: 6 x 9
##   school_type     mean_total sd_total min_total p25_total median_total p75_total
##   <chr>                <dbl>    <dbl>     <int>     <dbl>        <dbl>     <dbl>
## 1 Berufsbildende~       6.69     7.82         0      2            3.5      7.25
## 2 Erwachsenensch~      11       NA           11     11           11       11
```

```
## 3 Förderzentrum         1.75     0.5          1      1.75          2        2
## 4 Grundschule           5.29     5.47         0      2             4        7
## 5 Gymnasium             8        6.57         1      4             4.5     12.5
## 6 Oberschule            7        6.99         0      2             4.5      9.5
## # i 2 more variables: max_total <int>, n_schools <int>
```

For the plot we also want a split view that keeps separate averages for machines and shops. This lets the reader see in one glance whether differences between school types are driven more by retail shops or by vending machines.

```
stats_split <- counts_per_school %>%
  group_by(school_type) %>%
  summarise(
    mean_machines = round(mean(n_machines, na.rm = TRUE),2),
    sd_machines   = round(sd(n_machines,   na.rm = TRUE),2),
    mean_shops    = round(mean(n_shops,    na.rm = TRUE),2),
    sd_shops      = round(sd(n_shops,      na.rm = TRUE),2),
    .groups       = "drop"
  )

head(stats_split)
```

```
## # A tibble: 6 x 5
##   school_type           mean_machines sd_machines mean_shops sd_shops
##   <chr>                         <dbl>       <dbl>      <dbl>    <dbl>
## 1 Berufsbildende Schule          1.88        2.33       4.81     5.78
## 2 Erwachsenenschule              5          NA          6       NA
## 3 Förderzentrum                  0.25        0.5        1.5      0.58
## 4 Grundschule                    1.18        1.74       4.11     4.19
## 5 Gymnasium                      2.25        2.38       5.75     4.74
## 6 Oberschule                     1.63        2.1        5.37     5.36
```

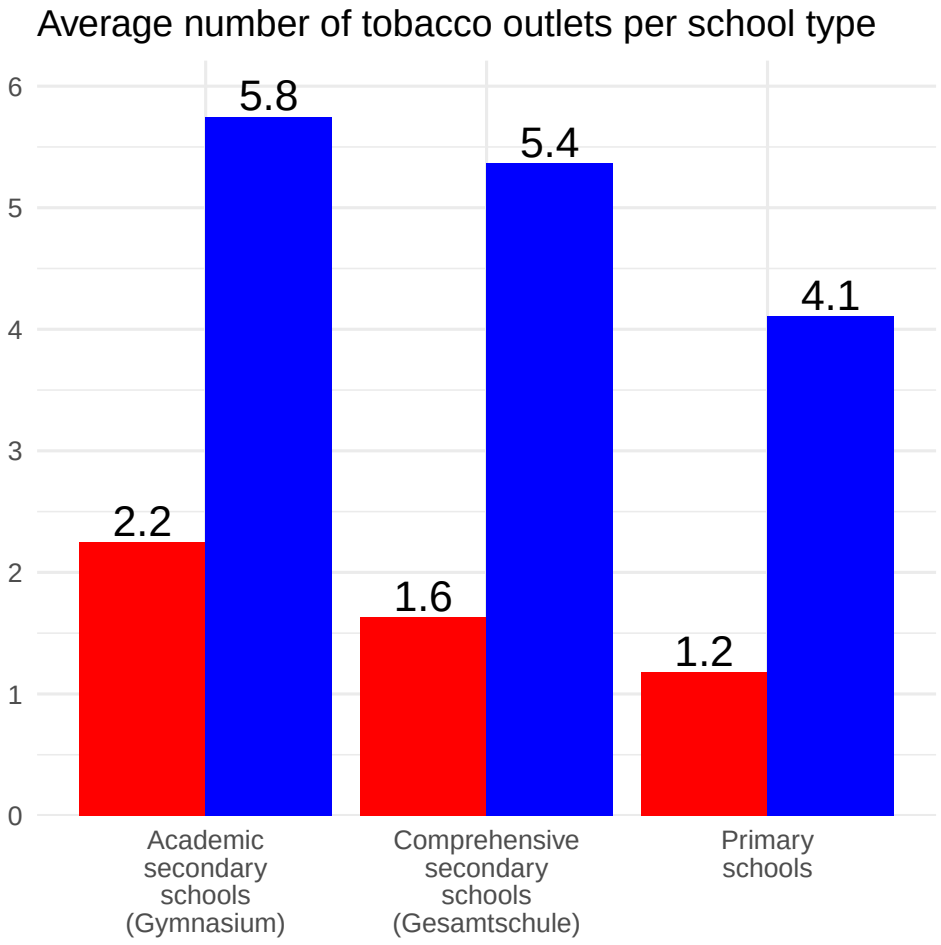
4.5 Bar

We now reshape the split table to a long format and draw a single bar chart that shows the average number of vending machines (red) and shops (blue) per school type. The labels help the reader read the values without hovering, and the wrapped x axis keeps longer labels readable. You can filter the school types to keep the focus on the ones that you are discussing in the text.

```
plot_split <- stats_split %>%
  pivot_longer(c(mean_machines, mean_shops),
               names_to  = "type",
               values_to = "mean_count"
               ) %>%
  mutate(type = recode(type,
                       "mean_machines" = "machines",
                       "mean_shops"    = "shops"))

plot_split %>%
  filter(school_type %in% c("Grundschule", "Gymnasium", "Oberschule")) %>%
  mutate(
    school_type_en = recode(school_type,
```

```
  "Grundschule" = "Primary schools",
  "Gymnasium"   = "Academic secondary schools (Gymnasium)",
  "Oberschule"  = "Comprehensive secondary schools (Gesamtschule)")
) %>%
ggplot(aes(x = school_type_en, y = mean_count, fill = type)) +
geom_col(position = position_dodge(width = 0.9)) +
geom_text(aes(label = round(mean_count, 1)),
          position = position_dodge(width = 0.9),
          vjust = -0.2, size = 5.5) +
scale_fill_manual(
  values = c("shops" = "blue", "machines" = "red")) +
guides(fill = "none") +
scale_x_discrete(labels = function(x) str_wrap(x, width = 10)) +
scale_y_continuous(breaks = pretty_breaks(n = 6),
                   expand = expansion(mult = c(0, 0.08))) +
labs(title = "Average number of tobacco outlets per school type",
     x = NULL, y = NULL) +
theme_minimal(base_size = 12)
```



Part 5: A small multilevel model

In this part we move from maps and counts to a simple model, so that we can ask whether the number of outlets near a school is related to population density or to a social index of the area. We assume that schools located in the same neighborhood share context.

We treat the **number of outlets per school** as the outcome, we use **population density** and a **social index** as predictors, and we add a **random intercept for the neighborhood** so that schools in the same area can have similar baseline levels. We standardise the predictors, because this makes the coefficients comparable and helps the model converge more easily. We also check whether the variance is much larger than the mean, because this tells us whether a negative binomial model is more appropriate than a Poisson model. In our example the counts come from the 500-meter buffers, therefore we can use the table from Part 3 directly.

5.1 Read the contextual data and join

Here the population file ("pop_density_data.csv") contains a column called `pop_index_2023` and the social file ("social_index_data.csv") contains a column called `social_index`. Both indicators are calculated at the neighborhood level. Both tables have an area name that we join to the school table through the column `school_neighborhood`. We use the R-Package "glmmTMB" by Brooks et al. (2017) for this analysis.

```
pop_density_data  <- here("project_data_champion_award",
                          "sources",
                          "pop_density_data.csv")
social_index_data <- here("project_data_champion_award",
                          "sources",
                          "social_index_data.csv")

pop_density <- read_csv(pop_density_data)
soc_index   <- read_csv(social_index_data)

# Join data, social index and population density by neighborhood name
basis_analysis <- counts_per_school %>%
  left_join(pop_density, by = c("school_neighborhood" = "area2")) %>%
  left_join(soc_index,   by = c("school_neighborhood" = "area"))

# Quick check for missing columns
stopifnot(all(c("n_total", "pop_index_2023", "social_index") %in% names(basis_analysis)))
```

5.2 Standardise predictors

We convert both predictors to z scores, which puts them on a similar scale and makes the interpretation easier. The school counts stay unchanged.

```
basis_analysis <- basis_analysis %>%
  mutate(
    pop_index_z    = as.numeric(scale(pop_index_2023)),
    social_index_z = as.numeric(scale(social_index))
  )
```

5.3 Check dispersion

When the variance is much larger than the mean, a negative binomial model is usually a better choice than a Poisson model. The following three lines compute mean, variance, and their ratio for all schools, and then for primary schools as an illustration.

```
# All schools (buffers are 500 m by construction)
mean_all  <- mean(basis_analysis$n_total, na.rm = TRUE)
var_all   <- var(basis_analysis$n_total,  na.rm = TRUE)
ratio_all <- var_all / mean_all
print(mean_all); print(var_all); print(ratio_all)
```

```
## [1] 6.59204
```

```
## [1] 49.47274
```

```
## [1] 7.504921
```

And we can do the same including only primary schools:

```
# Primary schools only
data_prim  <- filter(basis_analysis, school_type %in% c("Grundschule", "Private Grundschule"))
mean_prim  <- mean(data_prim$n_total, na.rm = TRUE)
var_prim   <- var(data_prim$n_total,  na.rm = TRUE)
ratio_prim <- var_prim / mean_prim
print(mean_prim); print(var_prim); print(ratio_prim)
```

```
## [1] 5.685393
```

```
## [1] 34.49081
```

```
## [1] 6.066565
```

If the ratio is clearly larger than one, you can proceed with a negative binomial model. If the ratio is close to one, a Poisson model may be sufficient.

5.4 Fit the multilevel model

We use a random intercept for `school_neighborhood` because schools in the same neighborhood share unobserved conditions. The formula reads as follows: the expected number of outlets depends on the social index and on population density, and there is a varying baseline by neighborhood.

```
model_nb <- glmmTMB(
  n_total ~ social_index_z + pop_index_z + (1 | school_neighborhood),
  data   = basis_analysis,
  family = nbinom2()
)
summary(model_nb)
```

```
##  Family: nbinom2  ( log )
## Formula:
## n_total ~ social_index_z + pop_index_z + (1 | school_neighborhood)
## Data: basis_analysis
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##    1059.4    1075.8    -524.7    1049.4       190
##
## Random effects:
##
## Conditional model:
##  Groups              Name        Variance Std.Dev.
##  school_neighborhood (Intercept) 0.2452   0.4952
## Number of obs: 195, groups:  school_neighborhood, 72
##
## Dispersion parameter for nbinom2 family (): 4.89
##
## Conditional model:
##                Estimate Std. Error z value Pr(>|z|)
## (Intercept)     1.56455    0.08398  18.630  < 2e-16 ***
## social_index_z -0.02871    0.08089  -0.355    0.723
## pop_index_z     0.51019    0.06841   7.458 8.81e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The summary shows you the direction and the strength of the associations on the log scale, together with standard errors and z values. A positive coefficient for `pop_index_z` means that the expected number of outlets increases with population density, after we account for neighborhood differences.

You can also refit the same specification for primary schools, so that you see whether the pattern looks similar for this important group.

```
model_prim <- glmmTMB(
  n_total ~ social_index_z + pop_index_z + (1 | school_neighborhood),
  data   = data_prim,
  family = nbinom2()
)
summary(model_prim)
```

```
##  Family: nbinom2  ( log )
## Formula:
## n_total ~ social_index_z + pop_index_z + (1 | school_neighborhood)
## Data: data_prim
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     452.1     464.4    -221.1     442.1        81
##
## Random effects:
##
## Conditional model:
##  Groups              Name        Variance Std.Dev.
##  school_neighborhood (Intercept) 0.01924  0.1387
## Number of obs: 86, groups:  school_neighborhood, 62
##
```

```
## Dispersion parameter for nbinom2 family (): 3.07
##
## Conditional model:
##                Estimate Std. Error z value Pr(>|z|)
## (Intercept)     1.55521    0.09888  15.728  < 2e-16 ***
## social_index_z -0.03551    0.08242  -0.431    0.667
## pop_index_z     0.46750    0.06798   6.877 6.12e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

To sum it up, this tutorial illustrated how spatial data can be obtained, processed, and made usable for visualization as well as for statistical analyses. You can easily adapt the workflow to other research questions, for example by searching for different types of objects in OpenStreetMap or by defining a different bounding box for another study area.

More importantly, this is not only a technical exercise: incorporating features of the physical environment into analyses allows health psychology to move towards a more holistic perspective. By linking behavioral and psychological outcomes to characteristics of the built environment, researchers can gain richer insights into the contexts in which health-related behaviors occur.

And in case you are still not sure whether spatial data is relevant to your research, here are some questions (adapted from Baur et al., 2014) that can be addressed using GIS and that may inspire ideas for your own health psychology projects:

1. How do people imagine, think about, and remember health-related spaces? Example: How do women in different neighborhoods perceive safety when walking to maternal health clinics, and does this perception align with actual spatial patterns of crime and lighting infrastructure?

2. How do communities create, modify, or lose access to health-promoting environments? Example: Which communities have lost access to green spaces over the past decade, and are these changes disproportionately concentrated in socially disadvantaged areas?

3. How do individuals experience and navigate environments that influence their well-being? Example: Can GPS data from mobile health apps reveal how often people choose safe walking routes compared to faster but riskier routes, and how does this influence daily physical activity levels?

4. How do people act or interact with others and with health-related artifacts in these spaces? Example: Do clusters of tobacco outlets near schools and workplaces reinforce habitual purchasing patterns among adolescents and commuters?

5. How are health-related resources or risks distributed across different spatial arrangements? Example: Using openly available OpenStreetMap data, can we map whether pharmacies are more common in affluent neighborhoods while fast-food outlets cluster in disadvantaged ones?

6. How do people move within and between spaces that matter for health? Example: How far do people usually need to travel to reach the nearest doctor's office, and how can we measure this in the same way across different cities to make results easier to compare?

References

Baur, N., Hering, L., Raschke, A., & Thierbach, C. (2014). Theory and methods in spatial analysis: Towards integrating qualitative, quantitative and cartographic approaches in the social sciences and humanities. Historical Social Research / Historische Sozialforschung, 39(2), 7–50. https://doi.org/10.12759/hsr.39.2014.2.7-50

Brooks, M. E., Kristensen, K., van Benthem, K. J., Magnusson, A., Berg, C. W., Nielsen, A., Skaug, H. J., Maechler, M., & Bolker, B. M. (2017). glmmTMB balances speed and flexibility among packages for zero-inflated generalized linear mixed modeling. The R Journal, 9(2), 378–400. https://doi.org/10.32614/RJ-2017-066

Müller, K. (2020). here: A simpler way to find your files (R package version 1.0.1). https://CRAN.R-project.org/package=here

Padgham, M., Rudis, B., Lovelace, R., & Salmon, M. (2017). osmdata. Journal of Open Source Software, 2(14), 305. https://doi.org/10.21105/joss.00305

Pebesma, E. (2018). Simple features for R: Standardized support for spatial vector data. The R Journal, 10(1), 439–446. https://doi.org/10.32614/RJ-2018-009

Pebesma, E., & Bivand, R. (2023). Spatial data science: With applications in R. Chapman & Hall/CRC. https://doi.org/10.1201/9780429459016

Tennekes, M. (2018). tmap: Thematic maps in R. Journal of Statistical Software, 84(6), 1–39. https://doi.org/10.18637/jss.v084.i06